Scripting Languages - Lecture 03

Lecture overview

Lecture 03 introduces core control flow mechanisms in scripting languages, specifically focusing on conditions and repetition. The session begins by defining conditional expressions that evaluate to Boolean values using operators like `==`, `!=`, and membership checks, emphasizing the distinction between equality testing (==) and variable assignment (=). These conditions drive the `if` statement, where indentation strictly defines code blocks executed only when a condition is true. The lecture expands on this with `else` and `elif` statements to handle multiple scenarios sequentially, as well as logical operators (`and`, `or`, `not`) for constructing complex conditions. It also clarifies Python's truthiness rules, where non-zero numbers and non-empty sequences evaluate to True, allowing for concise checks without explicit length comparisons.

The second half of the lecture covers repetition through `for`-loops and `while`-loops. `For`-loops iterate over sequences such as lists or strings, assigning each element to a variable in turn, and can be nested to process combinations of items or utilize the `range` function for fixed iterations. In contrast, `while`-loops repeat a code block based on a dynamic condition that must eventually become false to terminate execution. The session concludes with list comprehensions, a concise Python-specific syntax for generating new lists by iterating and filtering elements in a single line. Practical applications demonstrated include counting character occurrences, reversing strings, identifying palindromes via slicing, and checking for overlapping members between two lists.

Slide 1: faculty of arts

This lecture, titled "Conditions and repetition," is the third session in the Scripting Languages course for the Faculty of Arts. It is presented by Tim Van de Cruys on Wednesday, October 7, 2026. The slide serves as the title page introducing these specific topics without providing further technical details or definitions.

Slide 2: Today

Today's lecture covers four main topics in scripting languages: conditions, repetition, list comprehensions, and example problems. The section on conditions includes conditional expressions and the if-statement for conditional execution. Repetition is addressed through both for-loops and while-statements.

Slide 3: Making decisions: flow charts

This slide introduces the topic of making decisions using flow charts within the context of scripting languages. It serves as a section header for lecture material dated 3/24, presented by Tim Van De Cruys at KU Leuven. No specific definitions, examples, or detailed explanations regarding flow chart mechanics are provided in the available source text or aligned transcript for this slide.

Slide 4: Simple conditions

A conditional expression evaluates to True or False using operators like == (equality), `!=` (not equal), <, >, <=, >=, and `in` (membership). These results are Boolean values, named after George Boole. Crucially, the double equal sign (==) tests for equality, whereas a single equal sign (=) performs variable assignment.

Slide 5: Practical application: testing code

Conditions serve as a practical method to verify that code functions as expected by comparing results against anticipated values. For instance, checking if `3 * 3` equals `3 + 3 + 3` returns `True`, confirming the operation's correctness. This approach acts as a useful debugging strategy, encouraging developers to include such verification checks throughout their code to ensure it performs intended tasks.

Slide 6: The if-statement

The `if` statement executes indented code only when its condition evaluates to True, as shown in `if score >= 10: print('you passed the exam')`. In Python, indentation is obligatory and defines a code block-a sequence of lines following a colon-which determines exactly which statements are covered by the conditional. While four spaces are the standard convention for this indentation, it strictly replaces braces used in other languages to structure execution flow.

Slide 7: if-statement examples

This slide demonstrates a basic `if` statement in Python where a variable `score` is assigned the value 11. The conditional check `if score >= 10:` evaluates to true, triggering the execution of the print statement that outputs "you passed the exam". This example illustrates how scripting languages use indentation and comparison operators to control program flow based on specific conditions.

Slide 8: if-statement examples

This slide demonstrates two `if` statement examples in Python to evaluate exam scores against a threshold of 10. When `score` is set to 11, the condition `score >= 10` evaluates to true, printing "you passed the exam." Conversely, when `score` is 7, the condition fails, and no output is produced, illustrating how conditional logic controls program execution based on variable values.

Slide 9: if-statement examples

This slide demonstrates three `if`-statement outcomes based on variable types: an integer score of 11 triggers the print message, while a score of 7 results in no action. However, assigning the string `'12'` to the score causes an error because comparing a string object with an integer using a numerical operator is invalid. The example highlights that type consistency between the variable and the comparison value is essential for conditional logic to execute correctly.

Slide 10: Optional extensions of the if-statement

The `else` statement extends the `if` structure by executing its code block when the initial condition evaluates to false. Since conditions must be either true or false, this ensures that exactly one of the two options is necessarily executed. This approach allows for rearranging logic into a clear sequence of statements without requiring embedded conditional structures.

Slide 11: Optional extensions of the if-statement

The `if` statement can be extended with multiple extra conditions using `elif` (else if) statements to handle various scenarios sequentially. Conditions are evaluated from top to bottom; once a matching condition is found, its associated code executes and all remaining conditions are ignored. For example, checking if a score exceeds 16, then 10, or falls below allows for distinct outputs like "you did really well!" or "you passed the exam."

Slide 12: Complex conditions

Complex conditions are formed by chaining simple conditional expressions using the logical operators `and`, `or`, and `not`. The `and` operator returns True only if both conditions are true, while `or` returns True if either or both conditions are true. Additionally, `not` inverts a condition's truth value, as demonstrated by the example checking if a score is between 10 and 15 using `score >= 10 and score < 15`.

Slide 13: What counts as True and False?

In Python, the number zero and empty sequences are treated as False, while nonzero numbers and non-empty sequences are treated as True. Consequently, checking if a sequence is non-empty can be simplified by using the sequence directly in a conditional statement rather than explicitly comparing its length to zero. This preferred approach relies on Python's interpretation of non-empty objects as truthy values.

Slide 14: Converting a decision tree into if-statements

This slide demonstrates converting a decision tree regarding staying at a student reception into Python `if`-statements using Boolean variables `awkward` and `free_food`. The professor outlines three approaches: a literal nested structure checking conditions sequentially, an alternative nesting order, or a simplified version using logical operators like `and` and `not` to combine conditions. These methods illustrate how decision trees can be translated into readable code with varying levels of complexity.

Slide 15: Repeating statements: for-loops

This slide introduces `for`-loops in Python as a mechanism for repeating statements by iterating over items in a sequence, such as the list `['a', 'b', 'c']`. The provided example demonstrates that executing `print(item)` within the loop outputs each element sequentially. While the transcript contrasts this with `while` loops, which repeat until a condition is met, the slide specifically illustrates the syntax and result of iterating through a defined collection.

Slide 16: Repeating statements: for-loops

A `for`-loop repeats statements for every element in a sequence by assigning each element sequentially to a specified variable. In the example `for item in shoppinglist`, the variable `item` is assigned 'a', then 'b', and finally 'c' from the list, executing the indented block for each iteration. This mechanism efficiently handles iterative execution for lists and other sequences like strings, avoiding the need to manually write repetitive code for large datasets.

Slide 17: Looping over sequences

For loops iterate over any iterable object, such as strings or lists, by assigning each element sequentially to a variable. The slide demonstrates this by printing individual characters from the string 'word' and numbers from the list [0, 1, 2]. While the transcript mentions implementation details like indentation and variable naming, it primarily confirms that for loops provide an automatic way to process these sequences without manual index management.

Slide 18: Nesting loops inside loops

For loops can be nested, meaning an outer loop contains an inner loop that executes fully for each iteration of the outer loop. In the example `for number in [1, 2]: for letter in 'abc':`, the variable `number` remains constant while the inner loop iterates through all characters ('a', 'b', 'c') before `number` updates to the next value. This structure produces a sequence of pairs (1a, 1b, 1c, 2a, 2b, 2c) by systematically combining every element from the outer list with every element from the inner sequence.

Slide 19: Looping without a list

When repeating an action a specific number of times without using a list, the `range` function is utilized within a for loop. For example, `for number in range(4)` executes the loop body four times, printing the sequence 0, 1, 2, and 3. The final item is 3 because the counting mechanism starts at 0 rather than 1.

Slide 20: While-loop

The while-loop repeats a code block based on a condition, such as `countdown > 0`. Execution involves evaluating the condition; if true, the body runs and the loop restarts, but if false, execution exits to the next statement. In the countdown example, the loop prints numbers from 10 down to 1, decrementing by one each time, before finally printing 'Liftoff!' once the condition fails at zero.

Slide 21: List comprehensions

List comprehensions are a specific Python construction used to create a new list from an existing sequence, often utilizing a `for` loop to filter or transform elements. The slide illustrates this by filtering a list of fruits to retain only those containing the letter "a," demonstrating how a concise one-line comprehension achieves the same result as a traditional multi-line loop with conditional appending.

Slide 22: List comprehensions

List comprehensions are a Python-specific construction used to create a new list from an existing sequence by utilizing a for loop. This syntax condenses multiple lines of code, such as initializing a list and appending items based on a condition, into a single, more economical statement. For example, filtering a list of fruits to include only those containing the letter "a" can be expressed compactly as `[x for x in fruits if "a" in x]`.

Slide 23: List comprehensions

Slide 23 introduces list comprehensions as a scripting language feature. The slide provides the specific code example `[i for i in range(10)]` to demonstrate this syntax. No additional explanation or context is provided in the aligned transcript for this section.

Slide 24: List comprehensions

List comprehensions provide a concise syntax for creating lists in scripting languages. The slide demonstrates two forms: `[i for i in range(10)]` generates a list of numbers from 0 to 9, while `[i for i in range(10) if i < 5]` adds a conditional filter to include only values less than 5. These examples illustrate how iteration and optional filtering are combined within square brackets to construct lists efficiently.

Slide 25: List comprehensions

List comprehensions generate lists by iterating over a range and optionally applying conditions. The example `[i for i in range(10)]` produces numbers 0 through 9. Adding `if i < 5` filters the result to 0-4, while `if i % 2 == 0` selects only even numbers (0, 2, 4, 6, 8) from the original range.

Slide 26: List comprehensions

List comprehensions provide a concise syntax to generate lists by iterating over ranges or sequences. The examples demonstrate basic iteration, filtering with conditional statements (e.g., `if i < 5`), and transforming elements (e.g., `i*2`) within the same expression. These constructs allow for creating new lists efficiently without requiring explicit loop structures.

Slide 27: Example

This slide introduces a common programming task: counting the occurrences of a specific character within a string. Using the example word "banana," the objective is to determine how many times the letter 'a' appears. The process initializes a variable named `count` to zero, preparing it to accumulate the total number of matches found during the operation.

Slide 28: Reversing text

This slide introduces a programming challenge to write code that reverses a given string using a `for` loop. The specific example provided demonstrates transforming the input string `'I am testing'` into the reversed result `'gnitset ma I'`. The implementation begins by initializing the source string and an empty result variable to accumulate the reversed characters.

Slide 29: Palindromes

A palindrome is a word that reads identically forwards and backwards, such as "radar," whereas "radio" is not. To recognize palindromes in code, reverse the string using slicing syntax with steps of minus one and compare the result to the original. If the reversed string equals the original, the check returns True; otherwise, it returns False.

Slide 30: Overlapping lists?

This slide presents a problem requiring code to determine if two lists share at least one common member, using `list1 = [1, 2, 3]` and `list2 = [1, 4, 6]` as examples. The professor suggests initializing a Boolean variable `overlapping` to `False`, flipping it to `True` upon finding a match, and then checking its value. Alternatively, a shorter solution involves using an `if` statement to directly check if a specific element from one list is present in the other.

Slide 31: Practical session / reading

Practical session 2 requires students to access the lecture's code-along notebook on Toledo and re-run the examples at home. Additionally, students must read chapters 5 and 6 of the textbook Python for Everybody. These tasks reinforce the scripting language concepts covered in this section of the course.

Key points to remember

The second half of the lecture covers repetition through `for`-loops and `while`-loops. `For`-loops iterate over sequences such as lists or strings, assigning each element to a variable in turn, and can be nested to process combinations of items or utilize the `range` function for fixed iterations. In contrast, `while`-loops repeat a code block based on a dynamic condition that must eventually become false to terminate execution. The session concludes with list comprehensions, a concise Python-specific syntax for generating new lists by iterating and filtering elements in a single line. Practical applications demonstrated include counting character occurrences, reversing strings, identifying palindromes via slicing, and checking for overlapping members between two lists.